
Assessment and Redesign of ESI Data Center: Toward a Modern Scalable Architecture

BENZIADA Fares Abderraouf GRINE Abderrahmane HIMEUR Idris
DIAR Adem Abdelhafidh BENMESBAH Youcef ¹

¹ *École nationale Supérieure d'Informatique (ESI), Algiers*
Academic Year: 2025-2026

Abstract— Modern institutions face growing pressure to support AI-driven research, high-performance computing, and scalable cloud services within infrastructure originally designed for static workloads. This paper presents a complete assessment and redesign of the ESI data center, motivated by concrete limitations: STP link blocking, outdated hardware, non-scalable architecture, a single-GPU compute stack and no centralized monitoring. The proposed architecture replaces the legacy three-tier topology with a leaf-spine fabric driven by eBGP and VXLAN/EVPN, enforcing multi-tenant isolation across seven VRFs, with dedicated AI/HPC domains served by a lossless RoCEv2 fabric and a three-tier storage model. Deployment is automated through an Ansible single-source-of-truth pipeline and validated via simulation-based test cases covering BGP convergence, ECMP symmetry, VRF isolation, MLAG redundancy, and idempotency. Beyond ESI, this work serves as a practical reference and design catalogue for universities and enterprises seeking to modernize or build a data center under real operational and budgetary constraints.

Keywords—Leaf-Spine architecture, VXLAN/EVPN, BGP, Multi-tenancy, High-Performance Computing, Network Automation, GNS3, Absible.

I. INTRODUCTION

Modern data centers serve as the vital infrastructure for enterprise computing, shifting from rigid, legacy architectures to automated environments driven by Cloud computing and Artificial Intelligence (AI). Higher education institutions experience similar demands, as academic networks must handle highly diverse and fluctuating workloads. For École nationale Supérieure d'Informatique (ESI), the data center must reliably host critical services ranging from daily pedagogical activities and administrative platforms to resource-intensive scientific simulations and emerging AI computing clusters.

However, traditional legacy data center networks rely on a three-tier model (Core, Distribution, and Access) governed by Layer 2 protocols like Spanning Tree Protocol (STP) and Virtual Local Area Networks (VLANs). This traditional design presents severe structural limitations, including static manual configurations, slow deployment speeds, and restricted horizontal scalability. At ESI, these technical bottlenecks manifest during high-traffic operations, such as the orientation and registration period for new university graduates, or when provisioning isolated environments for student labs and advanced research. To support these expanding digital needs, a complete modernization of ESI's infrastructure is necessary.

To address these limitations, we propose a modern data center architecture tailored specifically to ESI's operational requirements. This design utilizes a physical Leaf-Spine topology characterized by a high-throughput, full-mesh IP

underlay network running external Border Gateway Protocol (eBGP) and Equal-Cost Multi-Pathing (ECMP). Over this physical fabric, we implement a virtual overlay using Virtual eXtensible Local Area Network (VXLAN) combined with BGP Ethernet Virtual Private Network (EVPN) to achieve secure, multi-tenant isolation across different academic and administrative groups. Furthermore, to ensure rapid, error-free provisioning, we utilize Ansible to automate the deployment and configuration pipeline within a network simulator environment.

The rest of this paper is structured as follows. Section 2 reviews the theoretical framework of modern data centers and conducts a comparative vendor study. Section 3 analyzes ESI's existing infrastructure and requirements. Section 4 details the proposed physical, logical, compute, security, and monitoring architecture. Section 5 describes the automated deployment strategy. Section 6 presents the validation test scenarios and results, and Section 7 concludes the paper. Moreover, there is an appendix, that goes a little bit deeply in the details of deployment of the leaf-spine fabric using GNS3 REST API.

II. THEORETICAL BACKGROUND AND STATE OF THE ART

A. Legacy Data Center Architectures

Traditional data center networks were designed primarily to support north-south traffic moving into and out of the data center infrastructure. These networks rely on a hierarchical

three-tier framework composed of Core, Distribution, and Access switches. Physical compute and storage servers connect directly to the Access layer switches.

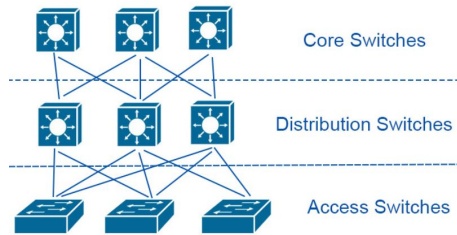


Fig. 1: Core-Distribution-Access Architecture.

This traditional design relies extensively on Layer 2 protocols, including STP and VLANs. However, with the rise of virtualization, distributed applications, and microservices, east-west traffic between internal servers has significantly increased. Consequently, traditional hierarchical architectures introduce major network bottlenecks, variable latency, and serious restrictions on overall performance and scalability.

B. Modern Leaf-Spine Fabric

To address traditional performance bottlenecks, the Leaf-Spine architecture reorganizes the data center network into a flattened, two-tier switching topology. The design features a Leaf layer consisting of Top-of-Rack (ToR) switches that provide access connectivity for compute and storage nodes. Above this sits the Spine layer, which functions as the high-speed backbone of the data center. Spine switches interconnect all leaf switches but never connect directly to servers. Every leaf switch maintains a physical link to every spine switch (full-mesh), creating a highly resilient structure where every leaf is equidistant from every spine.

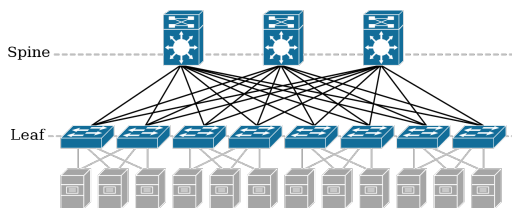


Fig. 2: Leaf-Spine Architecture.

Compared to legacy architectures, the Leaf-Spine fabric offers distinct operational advantages:

- **Predictable Performance:** The uniform two-hop design guarantees consistent latency across the entire fabric, which benefits heavy east-west traffic.
- **High Bandwidth Utilization:** Unlike legacy networks that rely on STP to block redundant links, modern fabrics leverage all physical paths simultaneously to maximize throughput.
- **Resilience:** If a spine switch fails, the remaining switches automatically distribute the load with minimal impact on performance, unlike legacy designs where a

distribution switch failure drops a large network segment.

C. Underlay Network Architecture

The underlay layer represents the physical IP fabric built on the Leaf-Spine infrastructure. It comprises physical switches, link connections, and routing protocols required to establish IP reachability between all nodes, providing a low-latency, highly available transport layer.

Modern designs widely adopt a BGP-based underlay due to its excellent scalability, flexibility, and incremental route updates. In the **eBGP model**, each switch belongs to a unique Autonomous System (AS) using private 2-byte AS numbers. Routing loops are natively prevented via the AS_PATH attribute, where routers reject any route containing their own AS number.

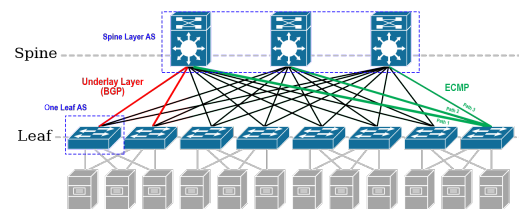


Fig. 3: Underlay Model (BGP & ECMP).

To maximize physical full-mesh capacity, Equal-Cost Multi-Path (ECMP) routing is enabled. ECMP distributes traffic across multiple equal-cost paths simultaneously by hashing a packet's 5-tuple (source IP, destination IP, source port, destination port, protocol), delivering high bandwidth utilization and fast convergence after failure.

D. Overlay Network Architecture

The overlay layer is a logical network built on top of the physical underlay, enabling virtualized Layer 2 or Layer 3 connectivity independent of the physical topology.

1. VXLAN

The data plane utilizes VXLAN encapsulation, where complete Layer 2 Ethernet frames are wrapped inside Layer 3 UDP packets to cross physical routing boundaries. Leaf switches function as VXLAN Tunnel Endpoints (VTEPs) managing encapsulation and decapsulation, while spine switches simply route the IP flows. VXLAN uses a 24-bit VNI, scaling capacity to 16 million isolated virtual networks, far surpassing the traditional 4,096 VLAN limit.

2. Multi-tenancy

Multi-tenancy is achieved through logical isolation: tenants are separated via Virtual Routing and Forwarding (VRF) instances for Layer 3 isolation and VNIs for Layer 2 segmentation. Under symmetric IRB, local VLANs map to unique L2 VNIs for isolated bridging, while routed traffic is handled inside per-tenant VRFs using L3 VNIs, securing isolation even with overlapping IP configurations.

3. EVPN Control Plane

Control plane operations are governed by BGP EVPN, which replaces traffic-based flood-and-learn MAC learning with deterministic, BGP-distributed forwarding updates. EVPN distributes endpoint locations (Type-2), BUM group membership (Type-3), and external routes (Type-5) over existing BGP sessions. This enables ARP suppression, which eliminates broadcast storms by answering requests locally at the ingress leaf, and seamless VM mobility through automatic BGP binding updates.

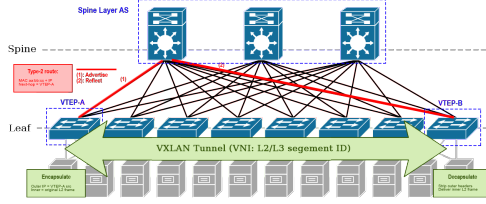


Fig. 4: Overlay Model (VXLAN & EVPN).

E. Scalability

The Leaf-Spine fabric scales incrementally and horizontally without requiring global network reconfiguration. For standard pod scaling, administrators add leaf switches to increase host port density or add spine switches to expand fabric bandwidth.

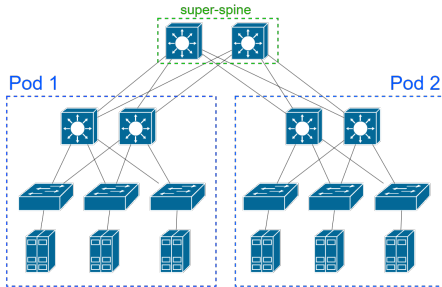


Fig. 5: Super-spine layer connecting two pods (4-hop cross-pod path).

When spine port counts are exhausted, a super-spine layer interconnects distinct pods, expanding the fabric diameter from 2-hop to 4-hop. Scaling limits are defined only by leaf TCAM capacity and spine BGP processing limits.

F. Vendor Comparative Study

Designing a modern data center requires evaluating enterprise hardware and software ecosystems to find the best fit for the deployment goals. Table 1 presents the key trade-offs across the four major industry vendors.

While Cisco and Huawei provide robust environments, they often impose strict licensing frameworks or geopolitical challenges. In contrast, Arista and Juniper offer superior API extensibility and structured operating systems that natively support multi-vendor environments and DevOps automation tools. Arista EOS is selected for this design based on its native streaming telemetry, first-class Ansible integration, and proven EVPN/VXLAN implementation.

III. ANALYSIS OF ESI'S EXISTING INFRASTRUCTURE AND REQUIREMENTS

A. Evaluation of the Current Data Center Architecture

ESI currently operates an on-premises data center organized around a traditional three-tier hierarchical topology. The existing networking environment utilizes Layer 3 switching and localized VLAN segmentation organized by individual buildings and server types. External connectivity is delivered through a dual Internet topology featuring a 1 Gbps primary fiber link backed up by a secondary satellite connection.

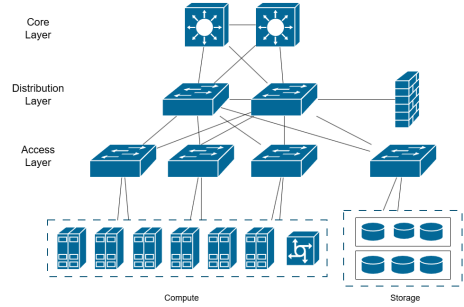


Fig. 6: High-Level Conceptual Topology for Current ESI Architecture.

The compute and storage infrastructure comprises 40 physical HP DL380 Gen servers virtualized via VMware ESXi hypervisors, running Intel Xeon processors with a maximum of 128 GB of RAM per server and only a single GPU across the entire stack. Storage is handled via two NAS systems with localized redundancy, and the environment operates with no active performance monitoring. Security relies on a Sophos software firewall paired with two dedicated physical firewalls. Table 2 summarizes the existing infrastructure.

Domain	Current State
Networking	3-tier Core/Distribution/Access; Layer 3 switching; VLAN segmentation per building; dual Internet (1 Gbps fiber + satellite backup)
Compute	~40 HP DL380 Gen servers; VMware ESXi; Intel Xeon; max 128 GB RAM/server; 1 GPU total
Storage	2 NAS systems; local redundancy; no NVMe-oF; no centralized storage management
Security	Sophos software firewall; 2 dedicated hardware firewalls
Monitoring	Local network monitoring only; no centralized performance or environmental monitoring

TABLE 2: SUMMARY OF THE CURRENT ESI DATA CENTER INFRASTRUCTURE.

B. Supported Services and Operational Demands

The existing data center hosts core academic, administrative, and national digital services:

- **Educational and Research:** Core academic platforms, VMs for student teaching environments and

Vendor	OS	Automation	Strengths / Considerations
Cisco	NX-OS	ACI / DCNM	Large ecosystem, strong enterprise support; high cost, complex licensing, proprietary ACI
Arista	EOS	CloudVision / AVD	Very stable OS, strong automation; smaller global footprint than Cisco
Juniper	Junos	Apstra	Structured OS, strong automation; industry uncertainty due to HPE acquisition
Huawei	VRP / CE-OS	iMaster NCE	Competitive pricing, good automation; geopolitical restrictions, complex licensing

TABLE 1: VENDOR COMPARISON FOR DATA CENTER NETWORKING.

Linux-based laboratories.

- **Administrative Applications:** Internal operational applications and institutional websites.
- **National Services:** BAC orientation platform, which experiences massive user traffic during specific peak periods.

C. Technical and Operational Limitations

A detailed technical assessment reveals several structural and operational constraints:

- **Hardware Degradation:** Outdated switching and server infrastructure limit system throughput, processing speed, and expansion capacity.
- **Architectural Non-Scalability:** The three-tier design is structurally rigid as expanding port density requires complex, manual reconfigurations of core layers.
- **Inadequate AI and Compute Resources:** A single GPU makes the environment completely unsuited for modern AI/ML workloads, with no dedicated HPC framework.
- **Storage Inefficiencies:** Existing NAS systems are not optimized for large-scale datasets required by scientific research.
- **Management and Visibility Gaps:** No centralized monitoring for network, system, or security configurations, with weak environmental tracking for temperature and hardware health.

D. System and Functional Requirements

To overcome these limitations, the redesigned data center must: ensure high availability of all existing academic, administrative, and national services; integrate a high-performance compute foundation for scientific simulations, deep learning, and large-scale dataset processing; deliver automated scalability with centralized resource management and unified monitoring; and enforce strict multi-tenant security segmentation, robust backup workflows, and comprehensive environmental telemetry.

IV. PROPOSED MODERN DATACENTER ARCHITECTURE

A. Design Rationale and Traceability

The proposed architecture directly addresses each identified limitation. The leaf-spine topology with ECMP resolves

STP link blocking and east-west bandwidth constraints. VXLAN/EVPN resolves the VLAN scale ceiling and enables automated provisioning. The VRF model provides per-domain isolation required by the service diversity. The dedicated AI/HPC leaf pair with lossless fabric resolves the GPU and RDMA gaps. Prometheus/Grafana and Zabbix/IPMI resolve the monitoring gaps. This explicit requirement-to-decision mapping distinguishes the proposed design from a generic leaf-spine template.

B. Physical Topology and Switch Selection



Fig. 8: Arista 7060DX5-64S Switch

The physical design uses a leaf-spine topology with full-mesh 400G uplinks between all spines and leaves, targeting a 1.23:1 oversubscription ratio for a near non-blocking fabric. Figure 7 shows the complete physical layout including the OOB management plane. Switch roles are mapped to Arista platforms as follows. Spine switches use the 7060DX5-64S (64×400G QSFP-DD, upgradeable to 800G; hardware DLB, PFC, RoCEv2). Border leaves use the 7050SX3-48C8C (48×SFP+ and 8×QSFP100) for WAN uplinks, inter-VRF routing, and firewall connectivity. General compute leaves use the 7050TX (48×10G-T and 6×QSFP+). AI/HPC leaves use the 7060DX5-64S, providing the same lossless non-blocking fabric as the spines with ECN and hardware RoCEv2 support. Storage leaves use the 7050X3 (8×200G QSFP28, upgradeable to 400G). The OOB management switch is the 7010T (48×1G and 4×10G) with IPMI/iDRAC connectivity over an isolated management network. MLAG is configured between each leaf pair to provide active-active host redundancy. All iDRAC and IPMI traffic is confined to the dedicated OOB plane, physically isolated from data and management plane traffic. Table 3 presents the initial compute and storage node inventory.

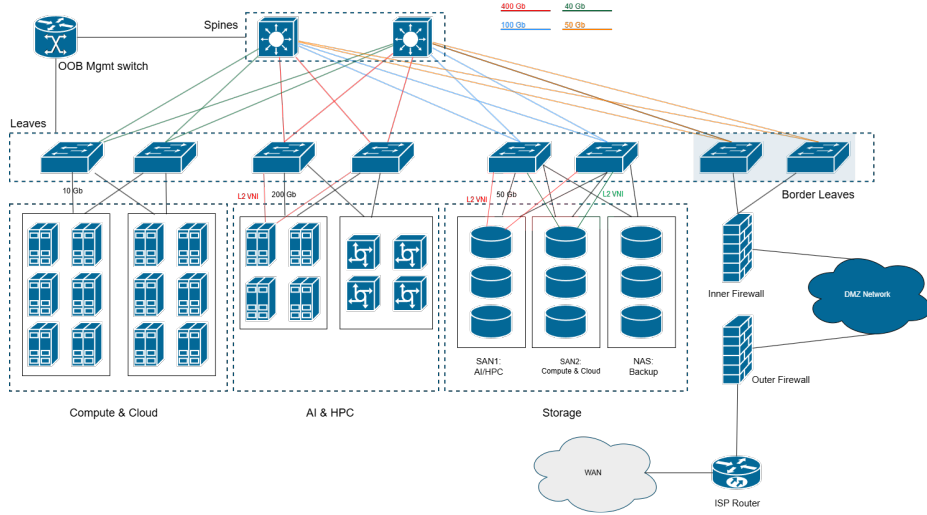


Fig. 7: Proposed physical topology: leaf-spine fabric with dedicated OOB management plane.

Node Type	Form	CPU / RAM / Storage	Qty
General Compute	2U	2× Xeon Scalable (16–24c), 256–512 GB DDR5, 2×NVMe + SAN	6
AI	4U	2× EPYC 9004 (32c), 512 GB–1 TB, 4×A100 80 GB, 4×NVMe + NVMe-oF	2
HPC	2U	2× EPYC 9554 (64c), 512 GB–2 TB, 2×NVMe + parallel FS	4
Storage	2U	1× Xeon (12c), 128 GB, 12×15 TB NVMe + 24×18 TB HDD (erasure)	3
Management	3U	1× Xeon (8c), 64 GB, 2×SSD RAID-1	2 (HA)

TABLE 3: INITIAL COMPUTE AND STORAGE NODE INVENTORY.

Domain	Key Services	Leaf Pair
General Cloud	BAC platform, Talents, Moodle, administrative apps, VM provisioning	LEAF-01/02
AI and HPC	GPU training jobs, AI labs, MPI simulations, parallel computing labs	LEAF-03/04
Storage	NVMe-oF block storage, NAS	LEAF-05/06
DMZ / Border	Public services, WAN uplinks, firewall connectivity	Border pair

TABLE 4: LOGICAL SERVICE DOMAINS AND DEDICATED LEAF PAIR ASSIGNMENTS.

D. Underlay Design

The underlay uses eBGP with per-switch ASNs: spine switches share AS65000, leaves use AS65001 through AS65006, and border leaves use AS65100. This flat ASN allocation avoids route reflectors, which introduce configuration complexity and a potential single point of failure in iBGP designs. IPv4 loopback addresses are exchanged via BGP extended next-hop encoding, enabling VTEP reachability across the fabric.

BFD is enabled on every fabric link with a 300 ms interval, 300 ms minimum receive interval, and detection multiplier of 3, achieving failure detection within approximately 900 ms. This triggers BGP to withdraw the failed path before the BGP hold timer expires, providing sub-second convergence independent of keepalive periods. ECMP is configured across all equal-cost spine paths.

E. Overlay Design

Each leaf acts as a VTEP. L2 forwarding uses per-tenant L2 VNIs and L3 forwarding uses per-VRP L3 VNIs under the symmetric IRB model. SVIs anchor each VLAN to its parent VRF, and a distributed anycast gateway presents a consistent gateway IP and MAC at every leaf, enabling VM mobility without IP address changes or gateway reconfiguration. BUM traffic uses ingress replication,

C. Logical Architecture and Service Domains

The logical architecture partitions services into four physical domains, each served by a dedicated leaf pair, as shown in Table 4 and Figure 9. This alignment of physical and logical boundaries prevents noisy-neighbor effects and simplifies per-domain QoS and security policy.

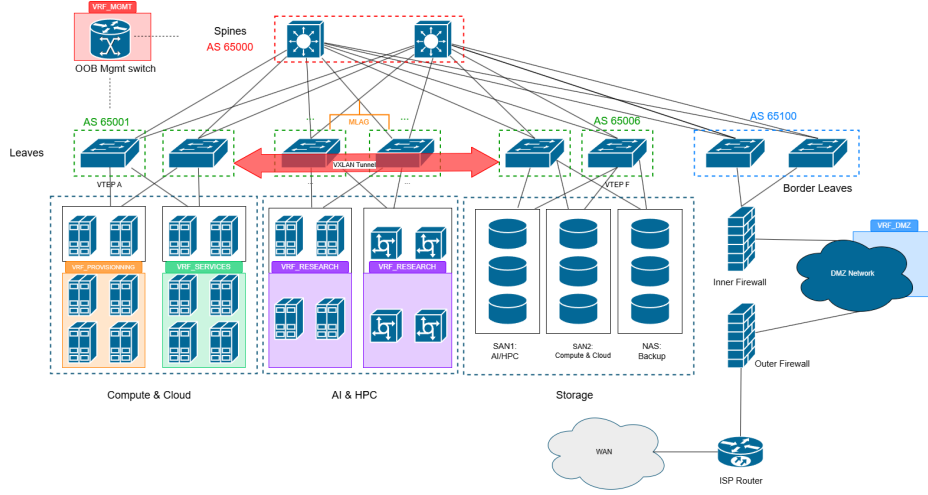


Fig. 9: Logical service domain layout across the leaf-spine fabric.

avoiding multicast dependencies in the underlay.

F. VRF Segmentation and Multi-Tenancy

The VRF overlay implements macro-segmentation by mapping each operational domain to a distinct trust boundary, restricting the network blast radius and enabling granular policy enforcement. Table 5 presents the complete VRF-to-L3VNI mapping.

SAN1 and SAN2 are L2-only VNIs that bypass Layer 3 routing entirely. Routing would introduce processing latency and, in the case of RDMA, break the lossless delivery guarantees that RoCEv2 requires. By keeping storage fabrics as pure L2 segments, RDMA semantics are preserved end-to-end. Inter-VRF routing follows a strict least-privilege model: shared services such as DNS and LDAP are centralized in VRF_SERVICES and accessed only through explicitly defined firewall policies; VRF_RESEARCH is reachable from other zones only through designated management bastion hosts; VRF_ADMINISTRATION is fully isolated from compute and public zones.

G. QoS Design

The fabric’s 1.23:1 oversubscription ratio and ECMP load balancing minimize structural congestion. QoS is applied selectively only where workload characteristics demand it. Table 6 defines the traffic classes.

Ingress leaves mark traffic using DSCP; the mark is preserved inside VXLAN headers across the fabric and re-marked at border leaves for WAN handoff. ECN signals early congestion, DCQCN reduces sender rates in response, and PFC is the last resort—confined to server-facing ports on AI/HPC and storage leaves and never propagated to spine switches. Lossless treatment applies exclusively to RDMA flows.

H. Storage Architecture

The storage design avoids a single-tier approach by segmenting into three independent tiers. Table 7 defines each tier.

Tier	Protocol	Connectivity	L
SAN1 – AI/HPC	NVMe-oF RoCEv2	Pure on leaves. Lossless (PFC/ECN). Not routed — RDMA-safe.	L2
SAN2 – Virt.	NVMe-oF TCP	Pure on compute leaves. Standard Ethernet; no lossless requirement.	L2
NAS – General	NFS	L3 via VRF_SERVICES. Filtered at border leaf and inner firewall.	L3

TABLE 7: STORAGE TIER SPECIFICATIONS AND CONNECTIVITY MODEL.

SAN1 prioritizes extreme performance for AI training: direct storage-to-GPU RDMA transfers require a lossless fabric with PFC and ECN; mixing this traffic with general flows would break lossless guarantees. SAN2 prioritizes high availability for VM live migration using NVMe-oF/TCP, which tolerates standard Ethernet without RDMA semantics. General NAS is L3-routed through VRF_SERVICES with firewall-enforced access control.

I. Security Architecture

The security model is built on a DMZ-based, dual-vendor firewall boundary. An outer firewall handles perimeter filtering and DDoS mitigation while an inner firewall enforces default-deny policies toward all internal VRFs. Using switches from different vendors for the two firewalls eliminates single-vendor compromise risk. All inbound Internet traffic terminates in the DMZ and no direct path from the Internet to any internal VRF exists. Figure 10 shows the security topology.

VRF	L3VNI	Scope and Usage
VRF_MGMT	20002	Admin SSH/RDP, hypervisor management (Proxmox), monitoring agents, OOB devices
VRF_INFRA	10001	VM orchestration, live migration traffic, PXE boot provisioning
VRF_SERVICES	40004	AD/LDAP identity, DNS, DHCP, NTP, NAS file access, backup, supervision
VRF_RESEARCH	50005	GPU nodes (RoCEv2), HPC login bastions, research dataset access
VRF_DMZ	40005	3-tier public exposure: web front-end, application back-end, database tier
VRF_ADMINISTRATION	30003	HR, payroll, academic records, institutional management systems
VRF_PROVISIONING	60006	Student and researcher VMs, micro-segmented per class/group via security groups
SAN1 (AI/HPC)	N/A	L2-only VNI — NVMe-oF RoCEv2 storage for AI/HPC; lossless, not routed
SAN2 (Compute)	N/A	L2-only VNI — NVMe-oF/TCP storage for general compute and virtualization

TABLE 5: VRF-TO-L3VNI MAPPING, FUNCTIONAL SCOPE, AND STORAGE L2VNIs.

Class	DSCP	Mechanism	Scope and Rationale
Network Control	CS6	Strict Priority (policed)	All switches: BGP, BFD, EVPN — must not be delayed or dropped
AI↔AI RDMA	AF41	Dedicated RDMA queue	AI/HPC leaves: RoCEv2 sustained elephant flows
HPC↔HPC RDMA	AF31	Dedicated RDMA queue	AI/HPC leaves: ultra-low-jitter MPI communication
Storage RDMA	AF21	Dedicated RDMA queue	AI/HPC and storage leaves: NVMe-oF over RoCEv2
Interactive	CS2	High-weight WRR/DWRR	DMZ/border: portals, APIs, bastion SSH, schedulers
Best-Effort	BE	Fair-share WRR/DWRR	All switches: NFS, S3, NVMe/TCP, general east-west

TABLE 6: SELECTIVE QOS CLASS MODEL FOR THE PROPOSED FABRIC.

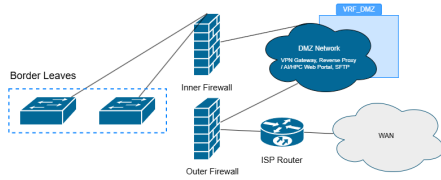


Fig. 10: DMZ-based dual-firewall architecture with VRF segmentation.

Authenticated access uses VPN for remote users, RBAC and MFA for administrative operations, and TLS/HTTPS/SFTP for all session encryption. DMZ-exposed services are limited to those explicitly required: a VPN gateway, a reverse proxy/API gateway, an AI/HPC web portal for job submission, a bastion host for administrative SSH, and an SFTP endpoint. All cross-VRF traffic is firewall-enforced; VRF segmentation alone is not treated as a security boundary without a stateful inspection point.

J. General Compute Cluster

The general compute cluster provides the virtualization and private cloud foundation for ESI’s day-to-day academic and administrative services. Six 2U nodes—each with dual Intel Xeon Scalable processors (16–24 cores), 256–512 GB DDR5, and local NVMe storage—attach to LEAF-01/02 and run KVM hypervisors managed by **Proxmox VE**. Proxmox SDN extends tenant networks directly over the BGP EVPN/VXLAN fabric, so new VMs inherit their VRF

assignment and VXLAN policy without any switch-level reconfiguration.

The cluster hosts three VM categories. Infrastructure VMs (DNS, DHCP, LDAP/AD, NTP, Backup) consume roughly 10% of cluster resources and are centralized in VRF_SERVICES. DMZ-facing VMs (Reverse Proxy, VPN Gateway, BAC Platform, AI/HPC Portal) hold approximately 25% inside VRF_DMZ, behind the inner firewall. Student and researcher VMs represent the dominant share at 65%, operating in VRF_PROVISIONING and micro-segmented per class or project group via Proxmox VE firewall rules to prevent lateral movement between tenant environments. During BAC orientation peaks, Proxmox VE resource pools reclaim idle student VM quota and redirect it to BAC platform replicas without any switch intervention. Shared block storage for VM live migration is provided by SAN2 (NVMe-oF/TCP, pure L2 on LEAF-01/02); general file access routes through VRF_SERVICES via the storage leaf pair, keeping live-migration traffic isolated from NFS at the routing layer.

K. AI/HPC Cluster

Placing AI and HPC workloads on the fabric required choosing between a converged design—co-locating both types on a single high-performance leaf pair (LEAF-03/04)—and dedicated silos, where each workload gets its own independently tuned leaf pair. Dedicated silos offer cleaner fault isolation and independent scalability, but at ESI’s current scale (two AI nodes and four HPC nodes)

they would leave the majority of 400G ports idle at disproportionate cost. The **converged model is therefore adopted**: QoS co-location risk is mitigated by reserving AF41 queues for AI-to-AI RDMA, AF31 for HPC-to-HPC MPI, and confining PFC to server-facing ports, never propagating it to the spine. Migrating to dedicated silos as demand grows requires only adding a second leaf pair to the same spine layer with no underlay redesign.

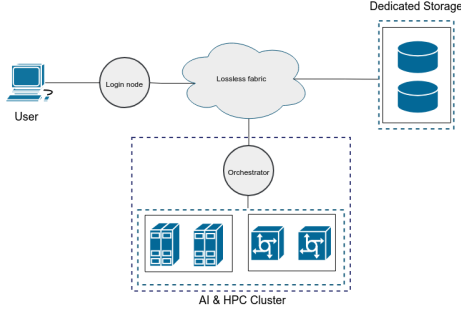


Fig. 11: AI/HPC cluster: login nodes, orchestrator, GPU/HPC compute nodes, and NVMe-oF storage over the lossless fabric.

Jobs are submitted through the AI/HPC web portal (DMZ-exposed) or via SSH through the bastion host to a login node. A **SLURM** orchestrator then schedules GPU or HPC workloads across compute nodes and allocates NVMe-oF volumes from SAN1. All inter-node communication—GPU gradient synchronization for distributed training or MPI message passing for simulations—traverses the LEAF-03/04 lossless fabric without crossing the spine when both endpoints share the same leaf pair. LEAF-03/04 runs the Arista 7060DX5-64S (identical to the spine platform), providing a non-blocking 400G fabric with hardware PFC, ECN, and RoCEv2; ECN triggers DCQCN rate reduction before any packet loss occurs, and PFC is the last resort, confined to server-facing ports. SAN1 operates as a pure L2 VNI—never routed—preserving RDMA semantics end-to-end from GPU node to NVMe target. The cluster operates within VRF_RESEARCH, fully isolated from all other domains; external access is gated exclusively through the AI/HPC portal and the bastion host.

L. Observability and Monitoring

To address the complete absence of centralized monitoring, a two-domain observability stack is introduced. In-band network and OS telemetry is collected by Prometheus and visualized through Grafana, aggregating real-time streaming telemetry from Arista EOS switches and compute nodes. Arista’s built-in streaming telemetry eliminates the need for polling-based SNMP. Out-of-band environmental and hardware monitoring uses Zabbix, interfacing with server IPMI/iDRAC over the dedicated 7010T OOB management switch to track thermal conditions, power draw, and hardware health independently of the data plane. This independence is critical: the OOB monitoring path must remain operational even if the data plane fails.

M. Scalability Strategy

Two concrete scaling phases are defined. *Phase 1 (Compute Scaling)*: Additional compute nodes connect to existing leaf pairs; Ansible automates integration and BGP EVPN ensures VM and container mobility without IP address changes, providing zero-downtime horizontal expansion. *Phase 2 (Fabric Scaling)*: A third spine switch is added, increasing total fabric bandwidth by approximately 50%. The 7060DX5-64S supports 800G interface upgrades, extending the fabric’s useful life to support next-generation AI accelerators (NVIDIA H100, B200) without hardware replacement. The border leaf pair is pre-configured to support future interconnection with peer institutions or a national AI research network.

V. AUTOMATED DEPLOYMENT

The deployment pipeline is implemented and validated in a GNS3 simulation environment. Figure 12 illustrates the full virtualisation stack, from the Python orchestration client at the top down to the physical host that accelerates every virtual node via KVM hardware virtualisation. This layered model is the execution context for all configuration and testing described in the subsections below.

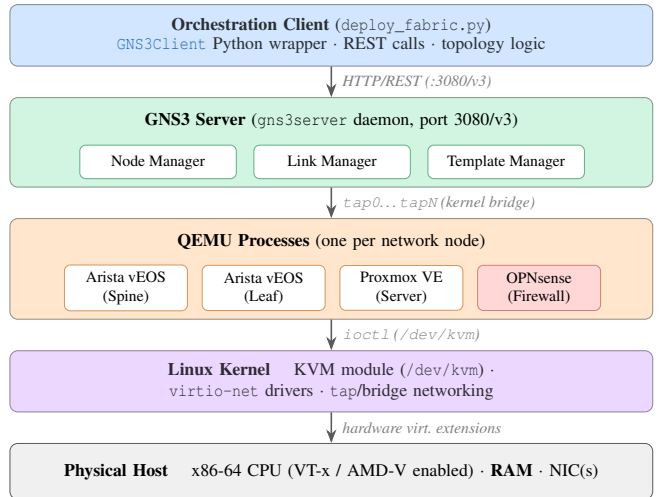


Fig. 12: Full virtualisation stack: from the Python orchestration client down to the physical host CPU. Each QEMU process represents one network node; KVM accelerates execution via hardware virtualisation.

A. Simulation Environment

The GNS3 v3 environment instantiates Arista vEOS nodes for spines and leaves, a Debian/FRRouting node for WAN-edge emulation, and an OPNsense node as the DMZ firewall. The baseline topology comprises two spines, five leaves (including the border pair), two server nodes, and the firewall; leaf pairs [L1,L2] and [L3,L4] form MLAG domains, and Border-1/Border-2 constitute the border pair. Management connectivity uses a 172.20.20.0/24 bridge with sequential IP assignment per node.

The simulation scope is bounded to three VRFs (VRF_PEDAGOGY, VRF_RESEARCH, VRF_DMZ), four tenant VLANs, and leaf ASNs starting at AS65100. The full seven-VRF mapping, dual-firewall topology, and storage tiers are specified in the design but not instantiated in the

simulator. This scoping allows the automation pipeline to be validated against realistic but manageable complexity.

B. Ansible Automation Architecture

Configuration is driven by two SSOT files—`sots/config.py` (topology: nodes, interfaces, ASNs, loopbacks) and `sots/vlans.py` (tenant: VLANs, VNIs, VRFs, route targets)—from which a dynamic inventory script generates four host groups (spines, border leaves, compute leaves, MLAG pairs), propagating any topology change to all playbooks automatically [1]. Four playbooks cover the lifecycle via the `arista.eos` eAPI collection: *underlay* (P2P, BFD, BGP, ECMP), *overlay* (VTEPs, VNIs, VRFs, SVIs, anycast GW, EVPN), *MLAG* (peer links), and *border* (WAN, inter-VRF). The excerpt below shows the declarative loop pattern driven by `fabric_interfaces`, which is SSOT-generated at inventory time:

```
- name: Configure fabric P2P interfaces with BFD
  arista.eos.eos_config:
    lines:
      - "description P2P to {{ item.peer_name }}"
      - "no switchport"
      - "ip address {{ item.ip }}/31"
      - "bfd interval 300 min-rx 300 multiplier 3"
      - "no shutdown"
    parents: "interface {{ item.interface }}"
  loop: "{{ fabric_interfaces }}"
```

C. Deployment Pipeline

The end-to-end pipeline is orchestrated by `launch.sh` in four sequential stages: (1) `topology/deploy_fabric.py` builds the GNS3 project, spawns nodes, and wires the full-mesh spine-leaf fabric via the GNS3 v3 REST API; (2) `configs/config_switches.py` bootstraps management IPs, eAPI, and SSH on each vEOS node; (3) `configs/config_frr.py` and `configs/config_firewalls.py` configure the FRR upstream router and OPNsense firewall respectively; (4) Ansible playbooks execute in dependency order—underlay, overlay, MLAG, border—each converging the target state idempotently. Re-running any playbook after a configuration change or node restart restores the desired state without operator intervention.

VI. VALIDATION AND TEST RESULTS

Each test case is executed by issuing EOS `show` commands over SSH to the target switch. The simulation instantiates a bounded scope (3 VRFs, 4 tenant VLANs, 2 spines, 4 leaves); tests therefore verify control-plane correctness rather than hardware throughput. Components outside this scope are noted explicitly.

TC-1 — BGP Underlay Convergence

`show bgp summary` on Border-1 (AS 65100) and Leaf-3 (AS 65102) shows all peers in Established with non-zero prefix counts (Fig. 13). Every leaf has formed adjacencies with both spines, and EVPN prefixes are exchanged over the

same sessions — confirming the unified underlay/overlay control plane.

Fig. 13: BGP summary on Border-1 (AS 65100) and Leaf-3 (AS 65102).

TC-2 — ECMP Path Symmetry

`show ip route 10.255.1.3` on Border-1 returns two equal-cost next-hops to the Leaf-3 loopback: via 10.0.0.0/Eth1 (Spine-1) and 10.0.0.8/Eth2 (Spine-2) (Fig. 14). Both uplinks carry traffic simultaneously — the fundamental contrast with the legacy STP design, where one path would be blocked.

Fig. 14: Route to Leaf-3 loopback on Border-1: two ECMP next-hops.

TC-3 — VTEP Reachability and Overlay Bootstrap

`show vxlan vtep` lists all three remote VTEPs (Border-2, Leaf-3, Leaf-4) as discovered peers; `show vxlan vni` confirms correct L3 VNI assignment per VRF (Fig. 15). Discovery occurs via BGP EVPN Type-3 (IMET) signaling — not data-plane flooding — and the symmetric IRB model has bootstrapped correctly.

Fig. 15: VTEP table (left) and VNI mapping (right) on Border-1.

TC-4 — EVPN Type-2 Control-Plane Readiness

`show bgp evpn summary` on Spine-1 shows all four leaf peers Established (Fig. 16). Prefix counts reflect each switch's role: Border-1 carries 30 prefixes and Border-2 carries 23 (inter-VRF leaking); Leaf-3 and Leaf-4 carry 8 each. No Type-2 MAC/IP routes are present because no host VMs are attached to access ports in the simulation; this is expected behavior and does not affect control-plane validity.

TC-5 — VRF Isolation Enforcement

`show ip route vrf VRF_PEDAGOGY` and `show ip route vrf VRF_RESEARCH` on Border-1 each show only local subnets plus a controlled set of symmetrically leaked routes (marked B L) (Fig. 17). Leaked routes steer inter-VRF traffic to the inner firewall for stateful inspection. The absence of a default cross-VRF route confirms that the least-privilege zone model is enforced at the forwarding plane.

- [9] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. W. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion control for large-scale RDMA deployments," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*, 2015. [Online]. Available: <https://conferences.sigcomm.org/sigcomm/2015/pdf/papers/p523.pdf>
- [10] A. S. George, "The evolution of data center networks: Strategies for modern infrastructure design," *ResearchGate Publication*, 2025.
- [11] Cisco Systems, "Out-of-band management best practices," Cisco Systems, Tech. Rep., n.d. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/collateral/service-provider/out-of-band-best-practices-wp.html>
- [12] Juniper Networks, "AI clusters data center design," Juniper Networks, Tech. Rep., n.d. [Online]. Available: <https://www.juniper.net/documentation/us/en/software/ncce/ai-clusters-data-center-design/ai-cluster-s-data-center-design.pdf>
- [13] FS.com, "How to build AI cluster networks with spine-leaf architecture and high-performance NICs," n.d. [Online]. Available: <https://www.fs.com/blog/how-to-build-ai-cluster-networks-with-spine-leaf-architecture-and-high-performance-nics-b47190.html>
- [14] NVIDIA, *DGX BasePOD Reference Architecture: Infrastructure Foundation for Enterprise AI*, NVIDIA Corporation, n.d. [Online]. Available: <https://docs.nvidia.com/dgx-basepod/reference-architecture-infrastructure-foundation-enterprise-ai/latest/reference-architectures.html>
- [15] GNS3 Technologies Inc., "Gns3 documentation," <https://docs.gns3.com>, 2024.
- [16] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, "KVM: the linux virtual machine monitor," in *Proceedings of the Linux Symposium*, vol. 1, 2007, pp. 225–230.
- [17] Arista Networks, "EOS central documentation," <https://eos.arista.com>, 2024.
- [18] Proxmox Server Solutions GmbH, *Proxmox VE Administration Guide*, 2024. [Online]. Available: <https://pve.proxmox.com/pve-docs/>

A. AUTOMATED SPINE-LEAF DEPLOYMENT VIA THE GNS3 REST API

This annex provides a self-contained guide to deploying a customised spine-leaf data-centre fabric using the `GNS3Client` Python wrapper and `deploy_fabric.py`. Readers wishing to adapt the code to a different topology need only modify `config.py`.

A. Prerequisites

1. **GNS3 Server 2.2+** running and reachable (default: `http://localhost:3080`).
2. **Appliance templates** pre-registered in GNS3: *Arista-vEOS*, *Proxmox VE*, *Debian Server*, *Opnsense* (names must match `config.py` exactly).
3. **Python 3.9+** with the `requests` library:

```
pip install requests
```

4. Network connectivity from the Python host to the GNS3 server HTTP API.
5. (Optional) For distributed deployments: additional hosts running `gns3server -compute` and registered as described in Section D.

B. Topology Configuration (`config.py`)

All topology and addressing parameters are defined in a single source-of-truth file. The key parameters are:

Parameter	Default	Description
NUM_SPINES	4	Number of spine switches
NUM_LEAVES	9	Number of leaf switches
NUM_SERVERS	4	Total server nodes (1 FRR + N-1 Proxmox)
MLAG_PAIRS	[[1, 2], [3, 4], [5, 6], [7, 8]]	MLAG groupings
FABRIC_SUBNET	10.0.0	Base for P2P /31 links
LOOPBACK_SPINE	10.255.0	Spine Loopback0 base
LOOPBACK_LEAF	10.255.1	Leaf Loopback0 base
VTEP_SUBNET	10.254.0	Leaf VTEP (Loopback1) base
SPINE_AS_BASE	65000	Shared spine BGP ASN
LEAF_AS_BASE	65100	Base for leaf BGP ASNs
MGMT_BASE_IP	172.20.20	Management subnet base
ENABLE_DMZ_FIREWALL	True	Deploy OPNsense firewall
COMPUTE_SPINES	["local"]	Spine host assignment list
COMPUTE_LEAVES	["local", "local"]	Leaf host assignment list
COMPUTE_SERVERS	["local"]	Server host assignment list
COMPUTE_FIREWALLS	["local"]	Firewall host assignment list

TABLE 9: KEY PARAMETERS IN `CONFIG.PY`.

To deploy a smaller 2-spine, 4-leaf topology with two MLAG pairs:

```
1 NUM_SPINES = 2
2 NUM_LEAVES = 4
3 NUM_SERVERS = 2          # 1 FRR + 1 Proxmox
4 MLAG_PAIRS = [[1, 2], [3, 4]]
5
6 COMPUTE_SPINES = ["local"]
7 COMPUTE_LEAVES = ["local"]
8 COMPUTE_SERVERS = ["local"]
9 COMPUTE_FIREWALLS = ["local"]
10
11 ENABLE_DMZ_FIREWALL = True
```

Listing 1: Minimal custom topology in `config.py`.

C. The *GNS3Client* Wrapper

1. Authentication

```
1 from gns3 import GNS3Client
2
3 gns3 = GNS3Client(
4     server = "http://localhost:3080/v3",
5     user = "admin",
6     password = "admin",
7 )
```

Listing 2: Authenticating with the GNS3 API.

Internally, `GNS3Client._authenticate()` posts credentials to `/access/users/login` and stores the returned JWT as a bearer token in the requests. Session headers, so all subsequent calls are authenticated automatically.

2. Project Management

```
1 project = gns3.create_project("MyDC")
2 project_id = project["project_id"]
3
4 for p in gns3.get_projects():
5     print(p["name"], p["project_id"])
6
7 gns3.delete_project(project_id)
```

Listing 3: Creating, listing, and deleting projects.

3. Spawning Nodes from Templates

```
1 templates = gns3.get_templates()
2 tid = next(t["template_id"] for t in templates
3           if t["name"] == "Arista-vEOS")
4
5 node = gns3.create_node_from_template(
6     project_id = project_id,
7     template_id = tid,
8     compute_id = "local", # or "compute-1", "compute-2", ...
9     x = 0,
10    y = -100,
11 )
12 node = gns3.rename_node(project_id, node["node_id"], "Spine-1")
```

Listing 4: Spawning and renaming a node from a GNS3 template.

4. Wiring Links

```
1 gns3.create_link(
2     project_id = project_id,
3     node_a = leaf1_id,
4     adapter_a = 1,
5     node_b = spine1_id,
6     adapter_b = 1,
7 )
```

Listing 5: Creating a link between two nodes.

The full fabric wiring loop:

```
1 next_adapter = {n["node_id"]: 1 for n in spines + leaves}
2
3 for leaf in leaves:
4     for spine in spines:
5         la = next_adapter[leaf["node_id"]]
6         sa = next_adapter[spine["node_id"]]
7         gns3.create_link(project_id,
8                           leaf["node_id"], la,
9                           spine["node_id"], sa)
10        next_adapter[leaf["node_id"]] += 1
11        next_adapter[spine["node_id"]] += 1
```

Listing 6: Full spine-leaf fabric wiring loop.

5. Management Switch and Cloud Node

```
1 mgmt_sw = gns3.create_node(
2     project_id, "MGMT-Switch", "ethernet_switch", "local",
3     x=800, y=-300,
4 )
5 gns3.set_switch_ports(project_id, mgmt_sw["node_id"], 24)
6
7 cloud = gns3.create_node(
8     project_id, "Cloud", "cloud", "local",
9     x=800, y=-400,
10    properties={"interfaces":
11        [{"name": "br0", "type": "ethernet", "special": False}]},
12 )
13
14 mgmt_port = 0
15 for node in spines + leaves + servers + firewalls:
16     gns3.create_link(project_id,
17                     node["node_id"], 0,
18                     mgmt_sw["node_id"], 0,
19                     port_b=mgmt_port)
20     mgmt_port += 1
21
22 gns3.create_link(project_id,
23                 mgmt_sw["node_id"], 0,
24                 cloud["node_id"], 0,
25                 port_a=mgmt_port)
```

Listing 7: Spawning and wiring the management plane.

6. Starting the Lab

```
1 gns3.start_nodes(project_id) # all nodes
2 gns3.start_node(project_id, spine1_id) # single node
```

Listing 8: Starting all nodes at once, or individually.

D. Distributed GNS3 Compute Infrastructure

A single physical host quickly becomes a bottleneck in large-scale emulations: Arista vEOS images alone consume approximately 2 GB of RAM each, and a 4-spine + 9-leaf fabric with four Proxmox servers totals roughly 35–40 GB of resident memory. GNS3 addresses this through its **multi-compute** architecture, which allows QEMU processes to be spawned on remote *compute servers* while the GNS3 main server continues to own the project, manage links, and serve the REST API.

1. Architecture Overview

In a distributed setup the GNS3 ecosystem comprises two roles:

- **GNS3 Server (controller):** owns the project state, exposes the REST API on port 3080, and orchestrates node placement. Runs on the primary host (also called *local* compute in `config.py`).
- **GNS3 Compute (worker):** a lightweight GNS3 daemon (`gns3server -compute`) running on each additional physical machine, listening on port 3080 (or a custom port). It receives instructions from the controller and manages QEMU processes locally.

Virtual links that cross compute boundaries are implemented as **VXLAN tunnels** between the GNS3 compute hosts. When a link is created between node A on `compute-1` and node B on `compute-2`, GNS3 automatically negotiates a UDP VXLAN tunnel between the two physical hosts, making the inter-machine link transparent to the emulated devices. This requires layer-3 reachability between all compute nodes; a flat management LAN or a dedicated interconnect switch is recommended. Figure A.1 illustrates a three-host distributed deployment as used in this project.

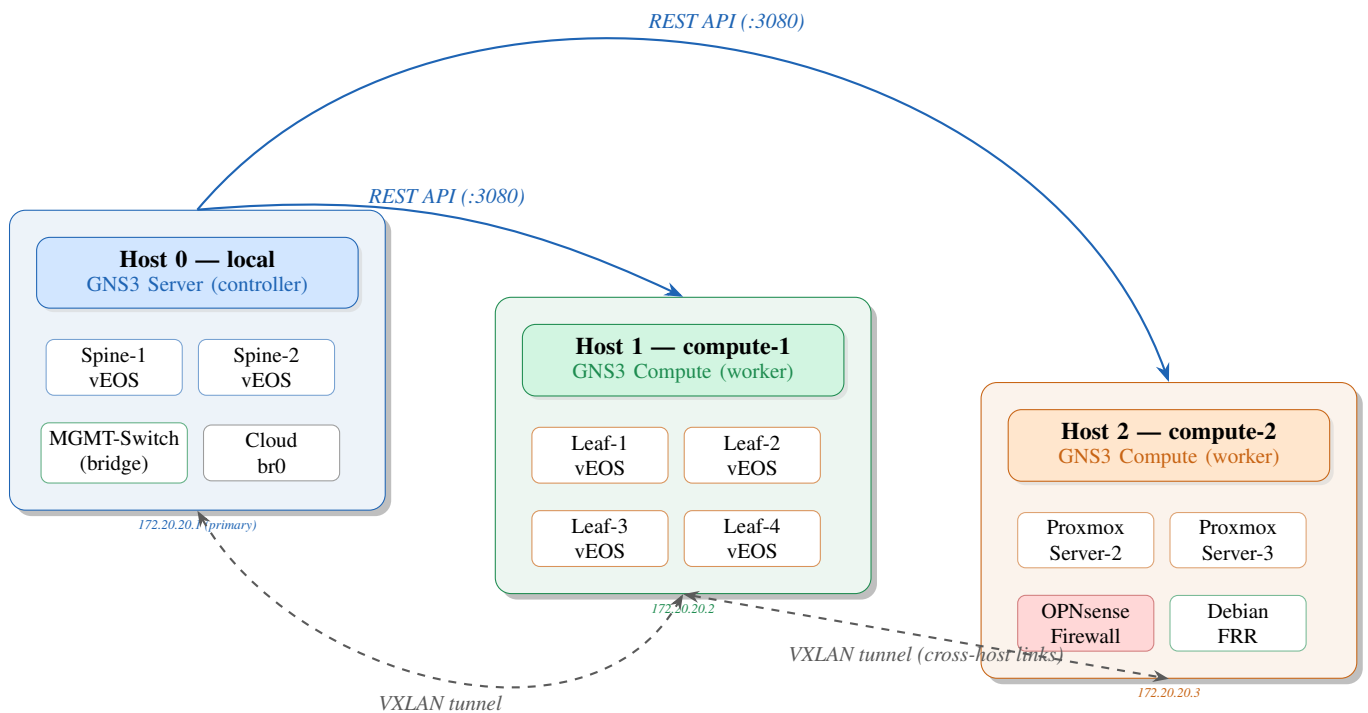


Fig. A.1: Three-host distributed GNS3 deployment. The controller (Host 0) owns the project and issues REST calls to the worker computes. Cross-host virtual links are transparently tunnelled over VXLAN UDP.

2. Registering Remote Compute Nodes

Before a remote host can receive nodes, its GNS3 compute daemon must be reachable from the controller. On each worker machine, the daemon is started as follows:

```
# Install GNS3 server (worker only needs the daemon, not the GUI)
pip3 install gns3-server

# Start the compute daemon, advertising its hostname/IP to the controller
gns3server --compute \
            --host 0.0.0.0 \
            --port 3080 \
            --username admin \
            --password admin
```

Listing 9: Starting the GNS3 compute daemon on a worker host.

On the controller, remote computes are registered through the web UI (*GNS3 Preferences* → *Server* → *Remote servers*) or via the REST API:

```
curl -X POST http://localhost:3080/v3/computes \
-H "Content-Type: application/json" \
-u admin:admin \
-d '{
    "compute_id": "compute-1",
    "name": "compute-1",
    "host": "192.168.1.101",
    "port": 3080,
    "user": "admin",
    "password": "admin",
    "protocol": "http"
}'
```

Listing 10: Registering a remote compute via the REST API.

Once registered, the compute name ("compute-1") becomes a valid token in `config.py`.

3. Configuring Node Distribution in `config.py`

Node assignment is controlled by three lists, one per device class. Each list is round-robin independently, allowing fine-grained load distribution:

```
1 # All spines on the primary host (memory-intensive, keep close
2 # to the GNS3 controller to reduce REST latency)
3 COMPUTE_SPINES = ["local"]
4
5 # Leaves split evenly across two worker machines
6 COMPUTE_LEAVES = ["compute-1", "compute-1",
7                  "compute-2", "compute-2"]
8
9 # Proxmox servers on the second worker (GPU / storage-rich)
10 COMPUTE_SERVERS = ["compute-2"]
11
12 # Firewall on the primary host (OPNsense is lightweight)
13 COMPUTE_FIREWALLS = ["local"]
```

Listing 11: Example multi-host compute assignment in `config.py`.

The round-robin assignment for a list $C = [c_0, c_1, \dots, c_{|C|-1}]$ and node index n (0-based within its class) is given by Equation 1:

$$\text{compute}(n) = C[n \bmod |C|] \quad (1)$$

For the leaf configuration above with $|C| = 4$, Leaf-1 goes to compute-1, Leaf-2 to compute-1, Leaf-3 to compute-2, Leaf-4 to compute-2, Leaf-5 back to compute-1, and so on across all $L = 9$ leaves.

4. Runtime Validation

The deployment script validates all compute names before spawning any node, preventing partial deployments:

```
1 computes = gns3.get_computes()
2 compute_map = {c["name"]: c["compute_id"] for c in computes}
3 compute_map["local"] = "local" # always available
4
5 def resolve(name):
6     if name in compute_map:
7         return compute_map[name]
8     raise RuntimeError(
9         f"Compute '{name}' not found. "
10        f"Available: {list(compute_map.keys())}"
11    )
12
13 # Validate all three lists up-front
14 compute_ids_spines = [resolve(n) for n in COMPUTE_SPINES]
15 compute_ids_leaves = [resolve(n) for n in COMPUTE_LEAVES]
16 compute_ids_servers = [resolve(n) for n in COMPUTE_SERVERS]
```

Listing 12: Compute resolution and validation in `deploy_fabric.py`.

5. Cross-Host Link Internals

When `GNS3Client.create_link()` is called for two nodes that reside on different compute hosts, the controller instructs both computes to create a shared VXLAN tunnel endpoint. GNS3 selects an ephemeral UDP port and records the `(host, port, VNI)` triple for each side. From the perspective of the emulated nodes the link is indistinguishable from a local tap-bridge connection; latency is dominated by the physical LAN between the two hosts. Table 10 summarises the memory footprint of each image type and the resulting capacity per physical host with 32 GB of RAM, assuming 2 GB of OS overhead.

Image	RAM / instance	Max instances	Notes
Arista vEOS	2.0 GB	15	All spines + leaves
Proxmox VE	4.0 GB	7	Nested KVM guests add overhead
Debian / FRR	0.5 GB	59	Minimal footprint
OPNsense	1.0 GB	30	Stateful NAT tables scale with sessions
Ethernet switch	<5 MB	—	Kernel bridge, no QEMU process

TABLE 10: APPROXIMATE QEMU RAM FOOTPRINT AND HOST CAPACITY (32 GB HOST).

E. GNS3 API Quick Reference

Method	Endpoint	Description
POST	/access/users/login	Obtain JWT bearer token
GET	/projects	List all projects
POST	/projects	Create a project
DELETE	/projects/{pid}	Delete a project
POST	/projects/{pid}/open	Open (activate) a project
POST	/projects/{pid}/close	Close a project
GET	/computes	List compute endpoints
POST	/computes	Register a remote compute
GET	/templates	List appliance templates
POST	/projects/{pid}/templates/{tid}	Spawn node from template
POST	/projects/{pid}/nodes	Create a custom node
PUT	/projects/{pid}/nodes/{nid}	Update node (rename, props)
GET	/projects/{pid}/nodes	List nodes in project
POST	/projects/{pid}/nodes/start	Start all nodes
POST	/projects/{pid}/nodes/{nid}/start	Start one node
POST	/projects/{pid}/links	Create a link
GET	/projects/{pid}/links	List links in project

TABLE 11: GNS3 REST API ENDPOINTS USED BY `GNS3CLIENT`.